

Práctica 4.1 Refactorización (CE4.a & CE4.b)



Nombre: Mateo Daniel Ortuño Ovando

| fecha de inicialización: 10/02/2026

Asignatura: Entornos de desarrollo

| fecha de finalización: 15/02/2026

INDICE

1. REFACTORIZACIÓN: EXTRACT METHOD Pág. 3

- 1.1. Análisis del código original y localización del bloquePág. 3
- 1.2. Resolución: Creación del método `calcularDescuento` Pág. 4

2. REFACTORIZACIÓN: RENAME METHOD Pág. 5

- 2.1. Evaluación de la claridad del método `imprimirDetalles` Pág. 5
- 2.2. Implementación: Aplicación de nombres descriptivos Pág. 5/6

3. REFACTORIZACIÓN: MOVE METHOD Pág. 7

- 3.1. Identificación de "Envidia de Funcionalidades" (*Feature Envy*) ... Pág. 7
- 3.2. Redistribución de responsabilidades entre `Circulo` y `Calculadora` .Pág. 7

4. CONCLUSIONES FINALES Pág. 8

1. Extract method.

Encuentra un bloque de código dentro del método que realice una tarea específica. Crea un nuevo método para ese bloque de código y haz que el método original llame al nuevo método en su lugar.

```
public void calcularPrecioTotal(int cantidad, double precioUnitario)
{
    double descuento = 0.1; double precioTotal = cantidad * precioUnitario;

    // Aplicar descuento si la cantidad es mayor a 100
    if (cantidad > 100)
    {
        precioTotal = precioTotal - (precioTotal * descuento);
    }

    // Imprimir el total
    System.out.println("Precio total: " + precioTotal);
}
```

1.1 extract method.

_Para extraer el metodo, descargo el proyecto, y me situo en el archivo de Clases.java y me voy a la linea numero 13 del codigo, y analizo el codigo de la clase el cual seria el siguiente:

```
public class Clases {

    public void calcularPrecioTotal(int cantidad, double precioUnitario)
    {
        double descuento = 0.1;
        double precioTotal = cantidad * precioUnitario;
        // Aplicar descuento si la cantidad es mayor a 100
        if (cantidad > 100)
        {
            precioTotal = precioTotal - (precioTotal * descuento);
        }

        // Imprimir el total
        System.out.println("Precio total: " + precioTotal);
    }
}
```

1.2 Resolucion el primer apartado.

_En la clase el primer apartado nos pide:

☐ ~~Encuentra el código del método que realice una tarea específica:~~

_este sería el método `calcularPrecioTotal()`;

☐ ~~Crea un nuevo método para ese bloque de código.~~

_Lo he creado y lo he llamado `calcularPrecioTotalNuevoMetodo()`

☐ ~~Hacer que el método original llame al nuevo en su lugar.~~

_Realizamos y llamamos al método original en su lugar.

```
public void calcularPrecioTotalNuevoMetodo(int cantidad, double precioUnitario)
{
    double descuento = 0.1;
    double precioTotal = cantidad * precioUnitario;
    //aplicar descuento si la cantidad es mayor a 100
    if (cantidad > 100)
    {
        // Aquí llamamos al nuevo método a diferencia del método anterior
        precioTotal = calcularDescuento(precioTotal, descuento);
    }
    System.out.println("Precio Total : " + precioTotal);
}
// Este es el método extraído que he creado
public double calcularDescuento(double total, double tasa)
{
    return total - (total * tasa);
}
```

_En la siguiente imagen, creamos el nuevo método, y llamamos al método que extrajimos, para calcular el descuento.

_ Ahora el código es más limpio porque el método principal ya no se preocupa por "cómo" se resta el descuento, solo sabe que debe pedirle ese resultado a `calcularDescuento()`.

2. Rename method.

Encuentra un método cuyo nombre no sea descriptivo o claro. Cambia el nombre del método a uno que refleje mejor su propósito y funcionalidad.

```
public void imprimirDetalles()  
{  
    // Lógica para imprimir detalles del producto  
    System.out.println("Detalles del producto:");  
    System.out.println("Nombre: Producto A");  
    System.out.println("Precio: $100");  
    System.out.println("Stock disponible: 50 unidades");  
    System.out.println("Proveedor: Proveedor X");  
}
```

2.1 Renombrar method.

_Para cumplir con el segundo objetivo de la practica analizamos la funcionalidad del metodo que nos dan, que seria el de imprimirDetalles(); el cual no devuelve nada porque es void.

```
public void imprimirDetalles() {  
    // Lógica para imprimir detalles del producto  
    System.out.println("Detalles del producto:");  
    System.out.println("Nombre: Producto A");  
    System.out.println("Precio: $100");  
    System.out.println("Stock disponible: 50 unidades");  
    System.out.println("Proveedor: Proveedor X");  
}
```

2.2 Realizamos el objetivo.

_Por lo que podemos ver el nombre no es el mas horrible que digamos, pero tiene fallos, que no es lo suficientemente especifico para usar debido a que “Imprimir detalles”, puede referirse a imprimir detalles de lo que sea, por lo que nuestro objetivo es que ningun programador en un futuro se confunda con ese metodo y hacerlo lo mas

especifico y entendible posible, por lo que he decidido, cambiar el nombre por el de:

```
public void mostrarFichaTecnicaProducto() {  
    // Lógica para imprimir detalles del producto  
    System.out.println("Detalles del producto:");  
    System.out.println("Nombre: Producto A");  
    System.out.println("Precio: $100");  
    System.out.println("Stock disponible: 50 unidades");  
    System.out.println("Proveedor: Proveedor X");  
}  
  
/**
```

_dejando asi el codigo mas util, y mejor organizado, con un nombre que sea acorde a lo que realiza.

3. Move method.

Encuentra entre las clases `Círculo` y `CalculadoraFiguras` un método que tenga más sentido en otra clase. Crea un nuevo método en la clase de destino y mueve el código relevante a ese método. Actualiza las llamadas al método para que utilicen la nueva ubicación.

3.1 Analizamos metodos de circulo y CalculadoraFiguras.

_El codigo de la clase circulo es este:

```
public class Circulo {
    private double radio;

    public Circulo(double radio) {
        this.radio = radio;
    }

    // Otros métodos y atributos de la clase Circulo

    // Método que utilizará el método calcularArea() que vamos a mover
    public void imprimirArea() {
        CalculadoraFiguras c= new CalculadoraFiguras();
        double area = c.calcularAreaCirculo(this.radio);
        System.out.println("El área del círculo es: " + area);
    }
}
```

_El codigo de la clase calculadoraFiguras es este:

```
package ejercicio3ud5_ed;

/**
 *
 * @author violi
 */
public class CalculadoraFiguras {
    public double calcularAreaCirculo(double radio) {
        return Math.PI * radio * radio;
    }
}
```

3.2 Realizamos move method.

_Como pudimos analizar, en la clase circulo esta la variable radio, en la clase calcularFiguras usamos el dato de radio, esto no tiene sentido, debido a que circulo sabe como calcular su propia Area.

_Por lo que hemos suprimido el metodo calcularAreaCirculo de la clase calculadoraFiguras, y lo hemos movido, a la clase circulo, asi nos ahorramos el uso de otra clase, que no tenia sentido que use un dato de circulo donde ya esta este dato, y en clase circulo implementamos el metodo calcularAreaCirculo.

```
public class CalculadoraFiguras {  
    // El método ha sido movido a la clase Circulo  
}  
}
```

_Nuestro codigo corregido quedaria asi:

```
// MÉTODO MOVIDO AQUÍ: Ahora es propio del círculo  
public double calcularArea() {  
    return Math.PI * this.radio * this.radio;  
}  
  
public void imprimirArea() {  
    // ACTUALIZACIÓN: Ya no necesitamos crear una CalculadoraFiguras  
    double area = calcularArea();  
    System.out.println("El área del círculo es: " + area);  
}  
}
```

4.Conclusiones finales.

_Hemos movido el codigo, y para que el compilador no se confunda con datos, aplicamos el this. Asi diferenciamos los atributos de la clase.

FIN.